COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	9/15/26	**DATE SUBMITTED:**	9/15/26
NAME:	Reyes, Cyrill Joferson B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 8.1 **Sorting Algorithms Pt2**

1. PROCEDURE OUTPUT

Code + Console Screenshot

```
const int SIZE = 100;

// Prints the array cleanly in 10 columns
void printArray(const int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << "\t";
        if ((i + 1) % 10 == 0) {
            std::cout << "\n";
        }
    }
    std::cout << "\n";
}
```

```
==================================================
 TABLE 8-1: ORIGINAL UNSORTED ARRAY (100 Elements)
==================================================
650     78      278     10      530     478     148     696     357     965
954     412     631     289     927     936     332     77      809     506
764     466     415     741     472     916     80      116     156     23
156     456     669     932     873     20      252     414     649     495
323     146     753     409     744     918     69      34      416     561
970     519     294     228     157     958     124     367     11      117
288     495     360     750     475     615     635     326     455     644
986     854     159     614     394     0       107     307     466     758
675     494     0       512     116     328     700     253     729     870
588     316     652     196     279     539     181     751     118     231
```

Observations

A set of 100 fake random numbers is added to an array using the rand() function. The initial value for the number was also provided by the time(0) function, and the final output shows a completely random collection of all the numbers from 0, 1, 2, and 3 until at least 999. This provides enough uniformity for testing each sorting method under the same conditions.

Table 8-1. Array of Values for Sort Algorithm Testing

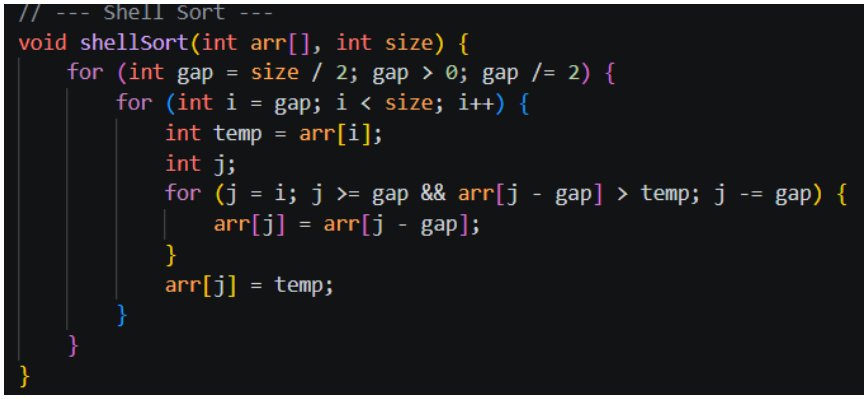
Code + Console Screenshot	 ========== TABLE 8-2: SHELL SORT RESULT ========== 0 0 10 11 20 23 34 69 77 78 80 107 116 116 117 118 124 146 148 156 156 157 159 181 196 228 231 252 253 278 279 288 289 294 307 316 323 326 328 332 357 360 367 394 409 412 414 415 416 455 456 466 466 472 475 478 494 495 495 506 512 519 530 539 561 588 614 615 631 635 644 649 650 652 669 675 696 700 729 741 744 750 751 753 758 764 809 854 870 873 916 918 927 932 936 954 958 965 970 986
Observations	Shell sort improves the standard insertion sort by comparing elements that are spaced apart by a changing gap.It starts with a gap of $N/2 = 50$ and then reduces the gap by half each time until it reaches 1. Early passes quickly move elements that are in the wrong place over long distances, which greatly reduces the need for shifting during the final pass when the gap is set to 1. The result shows that the list is fully sorted in ascending order and works efficiently with a time complexity between $O(N \log N)$ and $O(N^{1.5})$, depending on the sequence used for gaps.

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```
// --- Merge Sort Helpers & Implementation ---
void merge(int arr[], int left, int middle, int right) {
    int n1 = middle - left + 1;
    int n2 = right - middle;

    int* L = new int[n1];
    int* R = new int[n2];

    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[middle + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int middle = left + (right - left) / 2;
        mergeSort(arr, left, middle);
        mergeSort(arr, middle + 1, right);
        merge(arr, left, middle, right);
    }
}
```

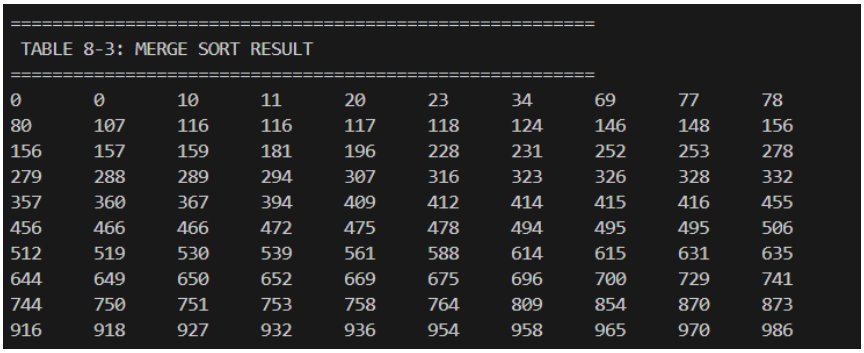
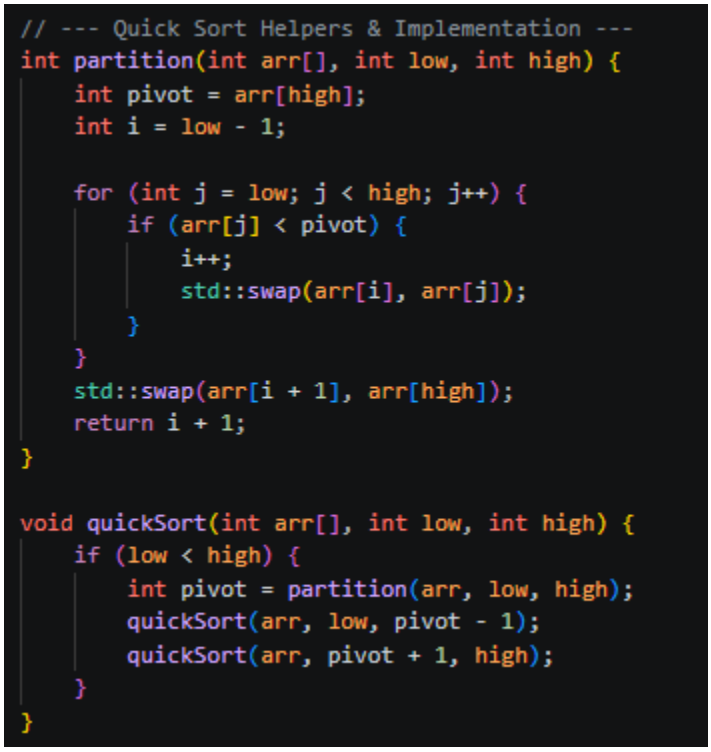
	
Observations	Merge Sort uses a method where it keeps breaking down a 100-element array into smaller parts, splitting it in half each time, until each part has just one element.Then, it puts those parts back together in order to create a fully sorted array. The algorithm ensures that both the worst-case and average time complexity remain $O(N \log N)$ no matter how the elements are arranged at the start.However, it needs $O(N)$ extra space for temporary arrays used during the merging process.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	

	``` == TABLE 8-4: QUICK SORT RESULT == 0 0 10 11 20 23 34 69 77 78 80 107 116 116 117 118 124 146 148 156 156 157 159 181 196 228 231 252 253 278 279 288 289 294 307 316 323 326 328 332 357 360 367 394 409 412 414 415 416 455 456 466 466 472 475 478 494 495 495 506 512 519 530 539 561 588 614 615 631 635 644 649 650 652 669 675 696 700 729 741 744 750 751 753 758 764 809 854 870 873 916 918 927 932 936 954 958 965 970 986 ```
Observations	Quick Sort splits the array by choosing a pivot element, which in this case is the last element, and then separates the elements into two smaller groups: one with elements smaller than the pivot and another with elements larger than the pivot. It works by handling elements directly without needing extra space, and it usually takes about O(N log N) time on average. The console output shows that the 100 elements are the same as the sorted results from the earlier algorithms.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

HOA8.1 > supplementary > main.cpp > main()

```cpp
#include <iostream>
#include "HybridSort.h"

void printArray(const int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << "\n";
}

int main() {
    int arr[] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19,
74};
    int n = sizeof(arr) / sizeof(arr[0]);

    std::cout << "Original Array: ";
    printArray(arr, n);
    std::cout << "----------------------------------------\n";

    hybridQuickSort(arr, 0, n - 1);

    std::cout << "----------------------------------------\n";
    std::cout << "Final Sorted Array: ";
    printArray(arr, n);

    return 0;
}
```

HOA8.1 > supplementary > C HybridSort.h > ...

```cpp
#ifndef HYBRID_SORT_H
#define HYBRID_SORT_H

#include <iostream>
#include <algorithm>

// Insertion Sort used to sort sub-lists
inline void insertionSort(int arr[], int low, int high) {
    for (int i = low + 1; i <= high; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= low && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

// Standard Quick Sort Partition
inline int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return i + 1;
}

// Quick Sort partition combined with Insertion Sort for left/right sub-lists
inline void hybridQuickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high);

        std::cout << "After Partitioning (Pivot = " << arr[pivotIndex] << "): ";
        for (int k = low; k <= high; k++) std::cout << arr[k] << " ";
        std::cout << "\n";

        // Sort Left Sub-list using Insertion Sort
        if (low < pivotIndex - 1) {
            insertionSort(arr, low, pivotIndex - 1);
        }

        // Sort Right Sub-list using Insertion Sort
        if (pivotIndex + 1 < high) {
            insertionSort(arr, pivotIndex + 1, high);
        }
    }
}

#endif
```

```
PS C:\Users\Jofjof\OneDrive\Documents\CPE12S4 - Reyes> cd "c:\Users\Jofjof\OneDriv
Original Array: 4 34 29 48 53 87 12 30 44 25 93 67 43 19 74
----------------------------------------
After Partitioning (Pivot = 74): 4 34 29 48 53 12 30 44 25 67 43 19 74 87 93
----------------------------------------
Final Sorted Array: 4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
PS C:\Users\Jofjof\OneDrive\Documents\CPE12S4 - Reyes\HOA8.1\supplementary> (Set-E
ivate.ps1")
(venv) PS C:\Users\Jofjof\OneDrive\Documents\CPE12S4 - Reyes\HOA8.1\supplementary>
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, debugging syntax errors, and concept explanation.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used as a reference to troubleshoot C++ compilation errors and clarify the O(N \log N) time complexity of sorting algorithms. All code was independently compiled, verified, and executed.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.